Wave-13 discovery-pressure audit — captured QA evidence (§11.4.83)

Revision: 1 **Last modified:** 2026-07-10T05:00:00Z

Run-id: `wave13-20260710`. Independent adversarial 2nd-pass audits (§11.4.118 loop-until-dry) of already-shipped code, each closure carrying real RED→GREEN + **conductor-run** polarity (§11.4.115) + independent gates (§11.4.125/§11.4.142/§11.4.134). Subagents ran zero git; the conductor is the independent review + polarity + commit seam (§11.4.20/§11.4.70).

Wave-13 is the convergence wave: defects are now rare and several independent passes return honest NO-DEFECT with enumerated coverage — the §11.4.118 loop-until-dry evidence that the known-issue set is approaching complete.

---

1. server API/middleware — truncated compressor on panic (commit `d15e7465`, main repo)

Defect: `compressionMiddleware` installed a body-compressing writer and closed it with a PLAIN post-`c.Next()` statement (`c.Writer = cw; c.Next(); _ = cw.Close()`). Production order is `recoveryMiddleware()` (OUTER) → … → `compressionMiddleware()` (INNER) → handlers. When a handler **panics**, the unwind SKIPS the post-`c.Next()` `_ = cw.Close()`; `recoveryMiddleware` then writes the 500 error envelope through the still-installed, never-finalised compressor, so a gzip/br-negotiating client receives `Content-Encoding: gzip` with a **truncated, undecodable** body — the error envelope never reaches the client (a §11.4 silent failure at the middleware layer).

Fix: close the compressor via `defer` (runs on every exit path incl. panic) and, when the compressor never engaged (`cw.cw == nil` — panic before first Write), restore the plain `ResponseWriter` so recovery's error is emitted intact. Happy path byte-identical (real response ⇒ `cw.cw` non-nil ⇒ writer left in place, single Close).

Files: `server/internal/api/compression.go` + `middleware_compression_test.go`.

Regression guard: `TestCompressionPanicErrorBodyDecodable` — drives the real production order (recovery + compression) with a panicking handler + `Accept-Encoding: gzip`, asserts status 500 AND the body is decodable per its declared `Content-Encoding` AND carries `CodeInternal`. Pre-fix: `unexpected EOF`. Post-fix: passes.

Independent gates (conductor): `go build`/`vet`/`gofmt` clean; `go test -race ./internal/api ./internal/transport` green.

Conductor polarity (§11.4.115): remove the writer-restore line inside the deferred closure (compile-safe — `cw`/`c` stay referenced) ⇒ on panic-before-write `cw.Close()` is a no-op and `c.Writer` stays the wrapper ⇒ recovery re-engages the compressor for the 500 which the already-run defer never closes ⇒ truncated gzip ⇒ the test FAILs ("500 error body could not be read/decoded (truncated compressor): unexpected EOF"); build compiled (real test-fail, not an §11.4.1 break); `compression.go` restored byte-identical (sha256); restored tree GREEN. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_compression.py`.

Published: `d15e7465` pushed **4/4** (github, gitlab, gitflic, gitverse — all OK).

---

2. docs_chain CLI — verify panics on a corrupt state.json (commit `394270e`, submodule)

Defect: `cmdVerify` was the ONLY `state.Load` caller that discarded the error (`st, _ := state.Load(statePath)`). `state.Load` returns `(New(), nil)` ONLY for a *missing* file; a read/parse error returns `(nil, err)`. So a corrupt/truncated state.json left `st == nil`, which `runner.Prepare` dereferences (`baseline := st.Hashes(c.Name)`) — a nil-receiver deref → SIGSEGV. `verify` is the CI/pre-build DRIFT GATE and state.json is

gitignored + regenerated, so a partial write on a crash is realistic: the gate would crash with a Go panic (exit 2) instead of running. The two sibling callers (`cmdSync`, `cmdReBaseline`) already checked the error.

Fix:
`st, serr := state.Load(statePath); if serr != nil || st == nil { st = state.New() }` — `verify` never consults the stored baseline anyway (it recomputes every derived node and compares fresh bytes to on-disk content), so the correct behavior on an unreadable state file is exactly the missing-file behavior: proceed with an empty (cold) baseline. Happy path (valid/missing state) byte-for-byte identical.

Files: `docs_chain/cmd/docs_chain/main.go` + `verify_corrupt_state_test.go`.

Regression guard: `TestCLI_Verify_CorruptState_NoPanic` — builds the real CLI, writes a non-parsing state.json, runs `verify`, asserts NO `panic:`, exit 0, and stdout contains `in-sync` (proves the drift check actually ran — can't pass on a no-op).

Independent gates (conductor): `go build`/`vet`/`gofmt` clean; `go test -race ./...` green (all 8 packages).

Conductor polarity (§11.4.115): flip the guard to `serr != nil && st != nil` (compile-safe — both vars stay referenced) ⇒ the cold-baseline fallback is skipped for a corrupt state ⇒ nil `*State` reaches `runner.Prepare` ⇒ the nil-deref panic returns ⇒ the test FAILs (exit 2, "invalid memory address or nil pointer dereference"); build compiled; `main.go` restored byte-identical (sha256); restored tree GREEN. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_docschain_verify.py`.

Published: `394270e` FF-pushed github+gitlab (docs_chain mirror set). Parent gitlink bump batched into the Wave-13 consolidation commit.

---

3. android agent — thrown transient port failure crashes instead of Retry (commit `4e9dd89`, submodule)

Defect: `OtaPollWorker.runCycle` called the three injected suspend ports (`client.pollForUpdate()`, `downloader.downloadToLocal()`, `verifier.verify()`) with ZERO exception handling, mapping only their RETURNED sealed transient outcomes (`PollResult.TransientError`,

`DownloadOutcome.Failed`) to `PollState.Retry`. But any real HTTP/IO-backed port (OkHttp/Ktor/java.net) THROWS `IOException`/`SocketTimeoutException` on a transient network blip rather than returning a typed transient value. That throw escaped `runCycle` uncaught; a `CoroutineWorker` maps an uncaught exception to `Result.failure()` — NO WorkManager backoff retry — the opposite of the file's own header invariant. A briefly offline device reports the update-check as a hard failure for that occurrence instead of retrying. Inconsistent by construction: a Downloader that RETURNS Failed → Retry, but the same downloader THROWING on the same drop → failure().

Fix: a private `suspend fun <T> ioOrNull(block: suspend () -> T): T?` in the companion object runs one port I/O step, RETHROWS `CancellationException` (cooperative WorkManager cancellation must never be swallowed), and converts any other `Throwable` to `null`. The three suspend port calls are routed through it with the existing `?: return PollState.Retry` idiom. The pure `PollStateMachine` transitions are NOT wrapped, so an illegal transition still fails loudly. Happy path unchanged.

Files: `submodules/ota-android-agent/android/.../poll/OtaPollWorker.kt` + `OtaPollWorkerRuntimeTest.kt` (+4 JVM tests, runBlocking, no device/Robolectric).

Regression guards: `poll/download/verify throwing a transient IOException degrades to Retry` (3 tests; download+verify also assert `deletedPath == null` — a thrown error is NOT a rejection, no artifact deletion) + `CancellationException from a port is not swallowed into Retry` (`assertThrows` — anti-bluff guard proving the fix is a targeted transient-handler, not a blanket catch-all).

Independent gates (conductor):
`gradle :core:test :android:compileDebugKotlin :android:testDebugUnitTest --rerun-tasks` → BUILD SUCCESSFUL, exit 0, 8/8 tests, 0 failures/errors.

Conductor polarity (§11.4.115): replace the `null` in `ioOrNull`'s `catch (transient: Throwable)` body with `throw transient` (compile-safe — Nothing-typed, valid in the `T?`-returning try expression) ⇒ the 3 IOException tests FAIL with `java.io.IOException` (uncaught transient reproduced), the CancellationException guard + all prior tests still PASS; build compiled (test-fail, not an §11.4.1 break); `OtaPollWorker.kt`

restored byte-identical (sha256); restored tree GREEN. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_android_pollworker.py`.

Published: `4e9dd89` FF-pushed origin (`de5d0e7..4e9dd89`, github — ota-android-agent single-remote mirror set). Parent gitlink bump batched into the Wave-13 consolidation.

---

4. ota-rollout-engine — halted rollout re-poll misreports its halt cause (commit `3284620`, submodule)

Defect: `Engine.Evaluate`'s idempotent terminal branch `case StatusHalted:` hardcoded `Reason: ReasonErrorThreshold`. But `decide()` halts for TWO distinct reasons — `ReasonErrorThreshold` and `ReasonPostBootFailed`. The FIRST `Evaluate` returns the correct reason, but every SUBSEQUENT re-`Evaluate` of an already-halted rollout (routine control-plane polling hits this no-op terminal branch) reported a post-boot-health halt as `error_threshold_breached`. `Decision.Reason` is documented "for audit/alerting" (verdict.go), so downstream alerting keyed on the reason (boot-failure runbook vs error-budget dashboard) is fed the WRONG classification on every re-poll. Not a safety-invariant violation — halt still wins, `DeviceStatus` stays `DeviceDeployFailed` — but a real correctness defect in a documented audit output field.

Fix: persist the halt cause on the HALT transition (`st.HaltReason = dec.Reason`) and in the terminal branch report `st.HaltReason`, falling back to `ReasonErrorThreshold` ONLY for legacy states persisted before the field existed (empty value). New additive `HaltReason Reason` field on `State` (Clone deep-copies scalar fields via struct copy). Happy path + first-evaluation reason unchanged.

Files: `submodules/ota-rollout-engine/{engine.go, types.go}` + new `terminal_halt_reason_test.go`.

Regression guards: `TestEvaluateTerminalHaltReasonReflectsPostBootFailure` (halt via PostBootHealthFailed; first reason == ReasonPostBootFailed, then 3× re-Evaluate asserts Action==Halt, DeviceStatus==DeviceDeployFailed, Reason==ReasonPostBootFailed) + `TestEvaluateTerminalHaltReasonReflectsErrorThreshold` (companion guard pinning the common case).

Independent gates (conductor): `go build`/`vet`/`gofmt` clean; `go test -race -count=2 ./...` green (root + chaos + stress, §11.4.50 deterministic).

Conductor polarity (§11.4.115): mutate the ActionHalt write `st.HaltReason = dec.Reason` → `st.HaltReason = ""` (compile-safe — field stays declared+assigned, `dec` stays used in the switch) ⇒ post-boot halt persists an empty reason ⇒ the terminal branch falls back to ReasonErrorThreshold ⇒ the post-boot test FAILs ("terminal halt reason = error_threshold_breached want post_boot_health_failed"); build compiled; `engine.go` restored byte-identical (sha256); restored tree GREEN. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_rollout_haltreason.py`.

Published: `3284620` FF-pushed origin (`23da1cd..3284620`, github — ota-rollout-engine single-remote mirror set). Parent gitlink bump in the Wave-13 consolidation commit.

---

5. server API — multipart spill temp-file leak (main repo, consolidation commit)

Defect: `handleUploadArtifact` (`handlers_artifact.go`) calls `c.MultipartForm()` (which invokes `Request.ParseMultipartForm(engine.MaxMultipartMemory)`) but never calls `MultipartForm.RemoveAll()`. When a file part exceeds the in-memory threshold, Go's `mime/multipart` spills it to an `os.CreateTemp(os.TempDir(), "multipart-*")` file; without `RemoveAll()` every spilled upload orphans a temp file — an unbounded disk leak / disk-DoS vector on a long-running server. Reading the bytes via `readFilePart`/`Open()` does NOT delete the spill file. (`grep RemoveAll` over `internal/api` returned nothing pre-fix.)

Fix: register `defer func(){ if c.Request.MultipartForm != nil { _ = c.Request.MultipartForm.RemoveAll() } }()` immediately after the successful parse (err == nil), covering every subsequent exit path. Minimal, no other behavior change.

Files: `server/internal/api/handlers_artifact.go` + new `handlers_artifact_multipart_test.go`.

Regression guard: `TestArtifactUploadCleansSpilledMultipartTempFiles` — forces spill-to-disk deterministically without a 32 MB body via

`router.MaxMultipartMemory = 1` + `t.Setenv("TMPDIR", t.TempDir())` (controlled empty spill dir), POSTs a valid admin-authed multipart upload, asserts 201 (parse+spill ran), then walks the spill dir and asserts ZERO leftover `multipart-*` files. Pre-fix: 1 leftover ⇒ FAIL. Post-fix: 0 ⇒ PASS.

Independent gates (conductor): `go build`/`vet`/`gofmt` clean; `go test -race ./internal/api` green (12.1s).

Conductor polarity (§11.4.115): delete the `defer … RemoveAll()` block (compile-safe — only a self-contained defer removed, `c` stays referenced) ⇒ the spill temp file is never removed ⇒ the test's post-handler walk finds 1 leftover ⇒ FAIL ("1 leftover 'multipart-*' file(s)"); build compiled; `handlers_artifact.go` restored byte-identical (sha256); restored tree GREEN. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_multipart_leak.py`.

Published: in the Wave-13 consolidation commit (main repo, pushed 4/4). This closes the HANDOFF tracked in the compression commit `d15e7465`.

---

6. Convergence — independent NO-DEFECT passes (§11.4.118 loop-until-dry)

Independent 2nd-pass adversarial audits that examined their scope in depth and returned honest NO-DEFECT with enumerated coverage (no fabricated defect — §11.4.6):

- **ota-artifact-validator** — converged (no in-scope defect this pass; the historical nil-reader panic already fixed at `707f876` in Wave-11a and re-verified GREEN).
- **ota-telemetry-schema** — NO-DEFECT. Pure schema + (de)serialization + pure health-derivation brick, no I/O. `go build`/`vet`/`gofmt`/`go test -race` all green; `FuzzDecodeBatch` 257,636 execs, zero panics/crashers on the untrusted-JSON decode entry point. Enumerated 8-point coverage (codec malformed-input, encode lossy/silent, HealthThresholds NaN/±Inf bypass [already fixed prior pass], Verdict halt-wins-over-advance safety invariant, DeriveHealth rate math, CountsByEvent map (un)marshal, enum completeness, concurrency). The one historical defect (NaN-threshold silent-accept) already remediated and holds under independent re-verification.

- **ota-protocol** — NO-DEFECT. Pure contracts/validation library (payload/validate/types/ enums), stdlib-only, no I/O, no concurrency, no package-level mutable state. `go build/vet/gofmt/go test -race` (root + chaos + stress) all green. Enumerated per-file + per-defect-class coverage: `ValidateSHA256` length-check-before-byte-scan (no index OOB), `parseHeaderInt` rejects overflow/non-numeric (never coerces to 0), `ParsePayloadProperties(nil)` nil-map-safe, `marshalEnum/unmarshalEnum` fail closed with no partial write, `HasUpdate()` stores an independent copy (aliasing test passes). Fuzz (`FuzzValidateSHA256`, `FuzzParsePayloadProperties`) cross-checked vs independent oracles + chaos-with-`recover()` prove no panic on empty/UTF-8/binary/overflow input. Prior hardening passes already closed the `isBlank` full-whitespace + whitespace-only-field gaps.

- **server/internal/config** — NO-DEFECT. Env-based control-plane config (config.go, 246 LOC). `go build/vet/gofmt/go test -race` green. Enumerated coverage of all 16 env reads: the `HELIX_ARTIFACT_PUBKEY` trust boundary is fail-closed by FACT (server.go adopts the config key ONLY if `len == ed25519.PublicKeySize`; `resolvePublicKey` returns `(nil,false)` otherwise → uploads rejected, no panic; key never comes from the request); every numeric parse rejects negatives + surfaces parse errors; `HELIX_TRUST_TLS_PROXY` bool default false (safe), never inverted, never inspects a request header; the `HELIX_TOKEN_SECRET` dev fallback is a deliberate, extensively documented MVP choice (§11.4.6 — not a defect, not changed). No silent-swallow, no nil-deref, no panic path.

- **server/internal/fabric** — NO-DEFECT. The `Registry` (registry.go, 207 LOC) is a stateless-after-construction façade: `repo/now` set once in `New`, read-only thereafter; no map/slice/lock in-package — all shared state lives at the already-RWMutex-guarded `internal/store` seam. Positive evidence: an 80-goroutine barrier-released `-race -count=3` probe hammering every method + an outcome oracle (no PASS-run with zero evidence) came back clean, then the probe was DELETED (no residue), tree byte-untouched. The `CompleteRun` PASS-requires-evidence TOCTOU was explicitly analyzed and found NOT violable (evidence is append-only — no removal path anywhere — so once ≥1 evidence is observed the invariant holds permanently; a racing `AttachEvidence` can only cause a benign false-reject, never a PASS-with-zero-evidence).

Convergence status: this round netted **5 real fixes** (compression, docs_chain, android, ota-rollout-engine, multipart-leak) and **5 independent NO-DEFECT passes** (ota-artifact-validator, ota-telemetry-schema, ota-protocol, config, fabric). The single fix from GENUINELY-fresh coverage (ota-rollout-engine, never previously 2nd-pass audited) confirms the value of continuing to widen coverage; the other four fixes came from deeper 2nd/3rd passes of already-touched areas. The §11.4.118 loop is measurably drying but NOT yet declared dry — the large `internal/deviceemu` (~2537 LOC) emulation surface and several containers/docs_chain/bridge packages remain un-deep-audited; a further targeted round is warranted before claiming the known-issue set complete.